BPTT算法详解：深入探究循环神经网络（RNN）中的梯度计算【原理理解】

by https://blog.csdn.net/qq_22841387/article/details/139283146

引言

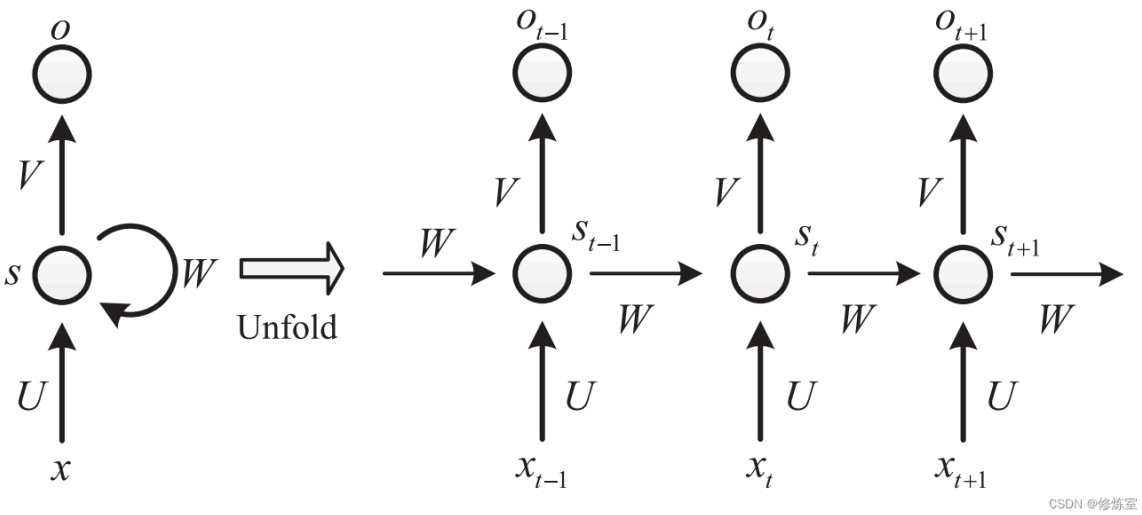
在深度学习 领域中，我们经常处理的是独立同分布的数据，比如图像分类、文本生成等任务，其中每个样本之间相互独立。然而，在现实生活中，许多数据具有时序结构，例如语言模型中的单词序列、股票价格随时间的变化、视频中的帧等。对于这类具有时序关系的数据，传统的深度学习模型可能无法很好地捕捉到其内在的 **时间相关性** 。为了解决这一问题，循环神经网络（Recurrent Neural Network, RNN）被广泛应用于处理时序数据。

> **为什么说反向传播算法不能处理时序数据呢?**
>
> 在传统的反向传播算法中，处理静态数据时，网络的输出 $\hat{y}$ 通常只依赖于当前时刻的隐藏状态 h，其更新规则可以表示为：
>
> $h = Wx + b$
>
> $\hat{y} = Vh + c$
>
> 其中，h 是隐藏状态，x 是输入，W 和 V 是网络的参数，b 和 c 是偏置项。
>
> 与传统反向传播算法不同，BPTT（Back-Propagation Through Time）算法引入了时间维度，并考虑了序列数据中的时序关系。在 BPTT 中，隐藏状态 h_t的更新规则包含了当前时刻的输入 X_t 和上一个时刻的隐藏状态 h_{t-1}，从而能够更好地捕捉到序列数据中的时间相关性。
>
> $h_t = f(UX_t + Wh_{t-1})$
>
> $\hat{y}_t = f(Vh_t)$

RNN 结构与BPTT

首先，让我们来了解一下常见的循环神经网络结构。在 RNN 中，隐藏状态会随着时间步的推移而更新，并在每个时间步生成一个输出。这种结构允许网络捕捉到序列数据中的时间相关性，使得其在时序任务中表现出色。

一个常见的RNN结构如下所示：



在RNN中，参数U、V和W是共享的，这意味着它们在每个时间步都保持不变。这意味着它们的值在整个模型 运行过程中 **始终保持一致** 。

BPTT算法概述

前向传播

在 RNN 中，前向传播阶段通过计算隐藏状态和输出来生成预测结果。

$$h_t = f(UX_t + Wh_{t-1})$$

$$\hat{y}_t = f(Vh_t)$$

损失函数

这些结果与真实标签之间的差异通过损失函数来衡量，我们的目标是最小化这个损失函数。整个网络的损失值L是每个时刻损失值L_t的求和，其中L_t是关于预测值$\hat{y}_t$的函数。

$$L_t = f(\hat{y}_t)$$

$$L = \sum_{i=1}^{T} L_t$$

> 损失函数 L 可以表示为：
>
> 1. **均方误差（MSE）：**
>
> $$L = \sum_{t=1}^{T} \frac{1}{2}(y_t - \hat{y}_t)^2$$
>
> 这里，我们计算每个时间步的输出 y_t 与真实输出 $\hat{y}_t$ 之间的平方误差，并将所有时间步的误差求和。
>
> 2. **交叉熵损失：**
>
> $$L = -\sum_{t=1}^{T} [\hat{y}_t \log(y_t) + (1 - \hat{y}_t) \log(1 - y_t)]$$

> 这里，我们计算每个时间步的输出 y_t 与真实输出 $\hat{y}_t$ 之间的交叉熵损失，并将所有时间步的损失求和。

反向传播

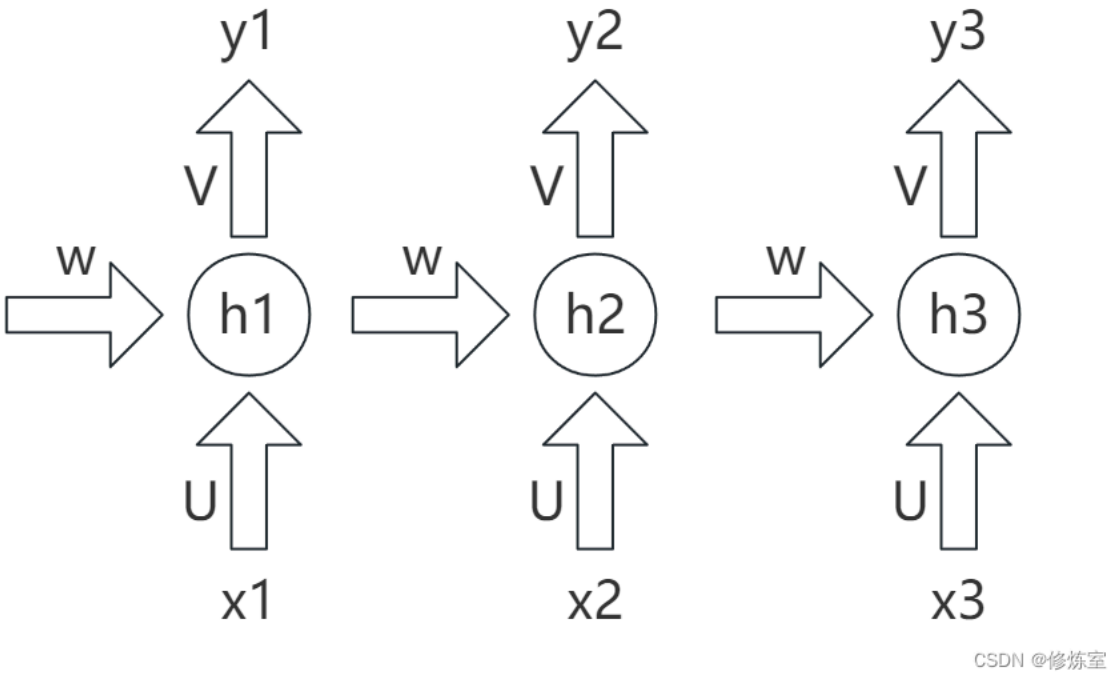
接下来，我们使用BPTT算法（随时间反向传播，Back-Propagation Through Time，BPTT）进行反向传播。在这一步中，我们计算损失函数对参数U、V和W的偏导数，以便更新参数以最小化损失。

> **为什么要使用整个序列的损失函数L对参数U、V和W求导呢?**
>
> 这是因为我们的目标是最小化整个序列的损失。在梯度下降算法中，梯度指向了损失函数增长最快的方向。因此，通过对整个序列的损失函数求导，我们可以找到在参数空间中使得损失函数逐步减小的方向，然后通过反向传播来更新参数。

由于RNN处理的是时序数据，因此需要基于时间进行反向传播，这也是BPTT名称的由来。尽管BPTT是在时序数据上进行反向传播，但本质上它仍然是反向传播算法，因此求解每个时间步的梯度是该算法的核心操作。

梯度计算

我们以一个长度为3的时间序列为例，展示对于参数U、V和W的偏导数的计算过程。



首先看看前向传播的计算

隐藏层输出：

$$h_t = f(UX_t + Wh_{t-1})$$

> **为什么“RNN的隐藏状态更新规则是 $h_t = f(UX_t + Wh_{t-1})$”?**
>
> 从数学角度来看，这个更新规则是由RNN的结构决定的。在RNN中，隐藏状态 h_t 是
>
> - 由当前时间步的输入 X_t
> - 前一个时间步的隐藏状态 h_{t-1}

> 组合而成的。通过线性变换 $UX_t + Wh_{t-1}$，加上激活函数 f 的作用，得到了新的隐藏状态 h_t。这个结构使得RNN能够记忆之前的信息并将其应用于当前的预测任务中。

输出层：

$$\hat{y}_t = f(Vh_t)$$

- h_t 是隐藏状态
- $\hat{y}_t$ 是输出值
- X_t 输入的序列
- f 是激活函数

将上面的RNN用数学表达式来表示就是

$$\begin{cases} h_1 = f(Ux_1 + Wh_0) \\ \hat{y}_1 = f(Vh_1) \end{cases}$$
$$\begin{cases} h_2 = f(Ux_2 + Wh_1) \\ \hat{y}_2 = f(Vh_2) \end{cases}$$
$$\begin{cases} h_3 = f(Ux_3 + Wh_2) \\ \hat{y}_3 = f(Vh_3) \end{cases}$$

针对$t = 3$时刻，求U，V，W的梯度（偏导），使用链式法则得到：

$$\frac{\partial L_3}{\partial V} = \frac{\partial L_3}{\hat{y}_3} \times \frac{\partial \hat{y}_3}{\partial V}$$

$$\frac{\partial L_3}{\partial W} = \frac{\partial L_3}{\partial \hat{y}_3} \times \frac{\partial \hat{y}_3}{\partial h_3} \times \frac{\partial h_3}{\partial W} + \frac{\partial L_3}{\partial \hat{y}_3} \times \frac{\partial \hat{y}_3}{\partial h_3} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_2}{\partial W} + \frac{\partial L_3}{\partial \hat{y}_3} \times \frac{\partial \hat{y}_3}{\partial h_3} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial h_1}{\partial W}$$

$$\frac{\partial L_3}{\partial U} = \frac{\partial L_3}{\partial \hat{y}_3} \times \frac{\partial \hat{y}_3}{\partial h_3} \times \frac{\partial h_3}{\partial U} + \frac{\partial L_3}{\partial \hat{y}_3} \times \frac{\partial \hat{y}_3}{\partial h_3} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_2}{\partial U} + \frac{\partial L_3}{\partial \hat{y}_3} \times \frac{\partial \hat{y}_3}{\partial h_3} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial h_1}{\partial U}$$

> 其实这个时候我们就可以看出，W和U两个参数的需要追溯之前的历史数据，参数V只需关注目前

所以，我们可以根据t3时刻的偏导，来计算任意时刻对U，V，W的偏导

对于V的偏导

对于V的偏导，我们直接将3替换成t即可：

$$\frac{\partial L_t}{\partial V} = \frac{\partial L_t}{\hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial V}$$

对于W的偏导

对于W的偏导，在t=3的时刻有三项，那么对应的在T时刻就有T项

$$\frac{\partial L_t}{\partial W} = \sum_{k=1}^{t} \frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_t} \times \frac{\partial h_t}{\partial h_k} \times \frac{\partial h_k}{W}$$

其中的$\frac{\partial h_t}{\partial h_k}$，我们可以进行展开：

例如在$k=1$时，$\frac{\partial h_3}{\partial h_1}=\frac{\partial h_3}{\partial h_2}\times\frac{\partial h_2}{\partial h_1}$

所以我们推导得到以下式子：

$$\frac{\partial h_t}{\partial h_k}=\frac{\partial h_t}{\partial h_k}\times\frac{\partial h_{t-1}}{\partial h_{t-2}}\times\ldots\times\frac{\partial h_{t-k+1}}{\partial h_k}$$

也就是等于：

$$\frac{\partial h_t}{\partial h_k}=\prod_{i=k+1}^{t}\frac{\partial h_i}{\partial h_{i-1}}$$

所以，

$$\frac{\partial L_t}{\partial W}=\sum_{k=1}^{t}\frac{\partial L_t}{\partial \hat{y}_t}\times\frac{\partial \hat{y}_t}{\partial h_t}\times(\prod_{i=k+1}^{t}\frac{\partial h_i}{\partial h_{i-1}})\times\frac{\partial h_k}{W}$$

对于U的偏导

同样的，我们也可以得到对于U的偏导

$$\frac{\partial L_t}{\partial U}=\sum_{k=1}^{t}\frac{\partial L_t}{\partial \hat{y}_t}\times\frac{\partial \hat{y}_t}{\partial h_t}\times(\prod_{i=k+1}^{t}\frac{\partial h_i}{\partial h_{i-1}})\times\frac{\partial h_k}{U}$$

> **为什么U也是这样的链式求导？**
>
> $$h_t=f(UX_t+Wh_{t-1})$$
>
> U也是通过链式法则求导的，因为隐藏状态h_t是由U、X_t和h_{t-1}共同决定的。因此，当我们计算损失函数关于U的偏导数时，需要考虑h_t对U的影响，而h_t又依赖于h_{t-1}，因此需要使用链式法则进行求导。

当前我们得到了是t时刻的导数，现在我们需要推广到整个网络中的损失值对U，V，W的偏导

总的损失值

因为和的导数等于导数，所以我们可以直接将$L=\sum_{i=1}^{T}L_t$前面的求和符号提出来

所以有，

$$\frac{\partial L}{\partial W}=\sum_{i=1}^{T}\frac{\partial L_t}{\partial W}$$

现在我们只需要将前面求得的t时刻的带入即可，

$$\frac{\partial L}{\partial W}=\sum_{i=1}^{T}\sum_{k=1}^{t}\frac{\partial L_t}{\partial \hat{y}_t}\times\frac{\partial \hat{y}_t}{\partial h_t}\times(\prod_{i=k+1}^{t}\frac{\partial h_i}{\partial h_{i-1}})\times\frac{\partial h_k}{W}$$

同样的，对于U，我们得到：

$$\frac{\partial L}{\partial U} = \sum_{i=1}^{T}\sum_{k=1}^{t}\frac{\partial L_t}{\partial \hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial h_t} \times (\prod_{i=k+1}^{t}\frac{\partial h_i}{\partial h_{i-1}}) \times \frac{\partial h_k}{U}$$

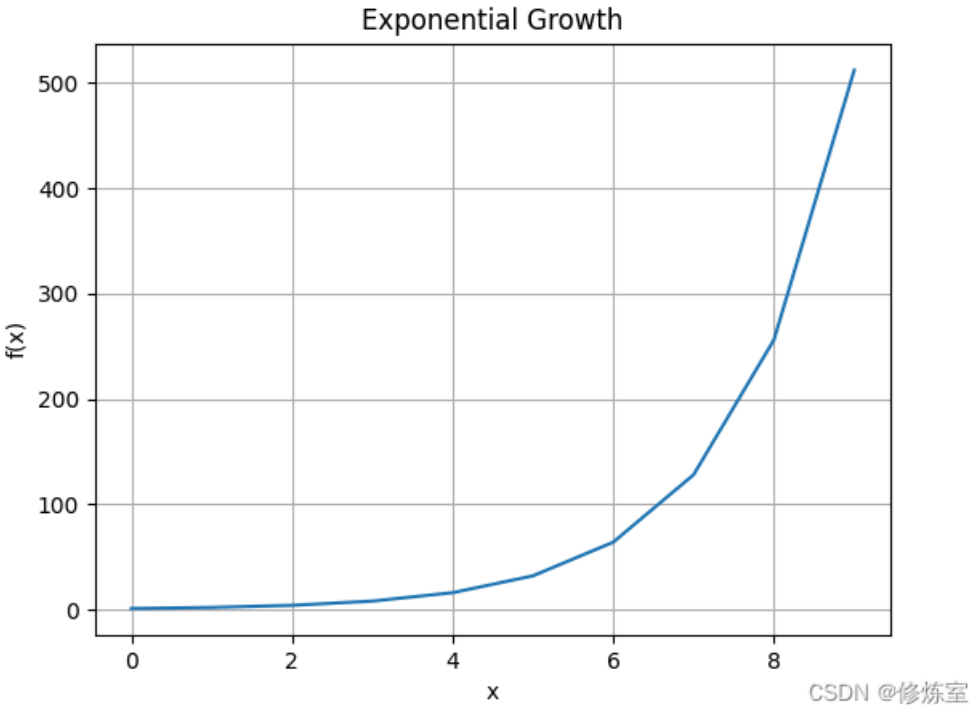
对于V，我们得到：

$$\frac{\partial L}{\partial V} = \sum_{i=1}^{T}\frac{\partial L_t}{\hat{y}_t} \times \frac{\partial \hat{y}_t}{\partial V}$$

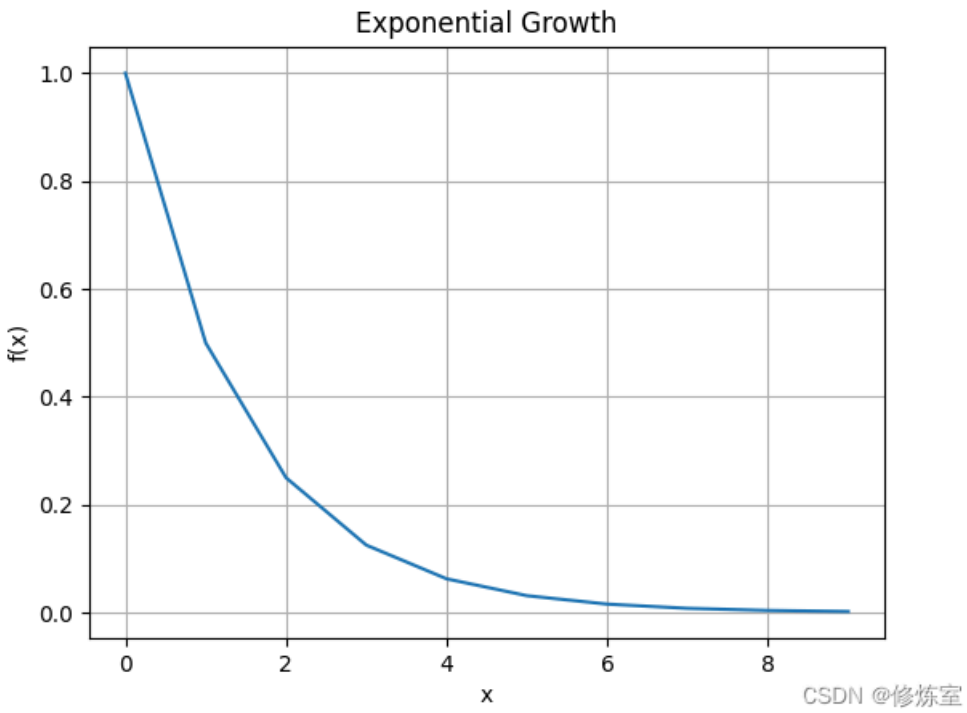
梯度爆炸和梯度消失问题

在W和U中，存在一个连乘 $\prod_{i=k+1}^{t}\frac{\partial h_i}{\partial h_{i-1}}$；也就是说，会出现指数级别的问题；

如果$\frac{\partial h_i}{\partial h_{i-1}}$的话，那么连乘的结果可能会快速增长，导致梯度爆炸。



如果$\frac{\partial h_i}{\partial h_{i-1}}$的话，连乘的结果会迅速衰减到零，导致梯度消失



我们来求解一下关于$\frac{\partial h_i}{\partial h_{i-1}}$数学上的表示：

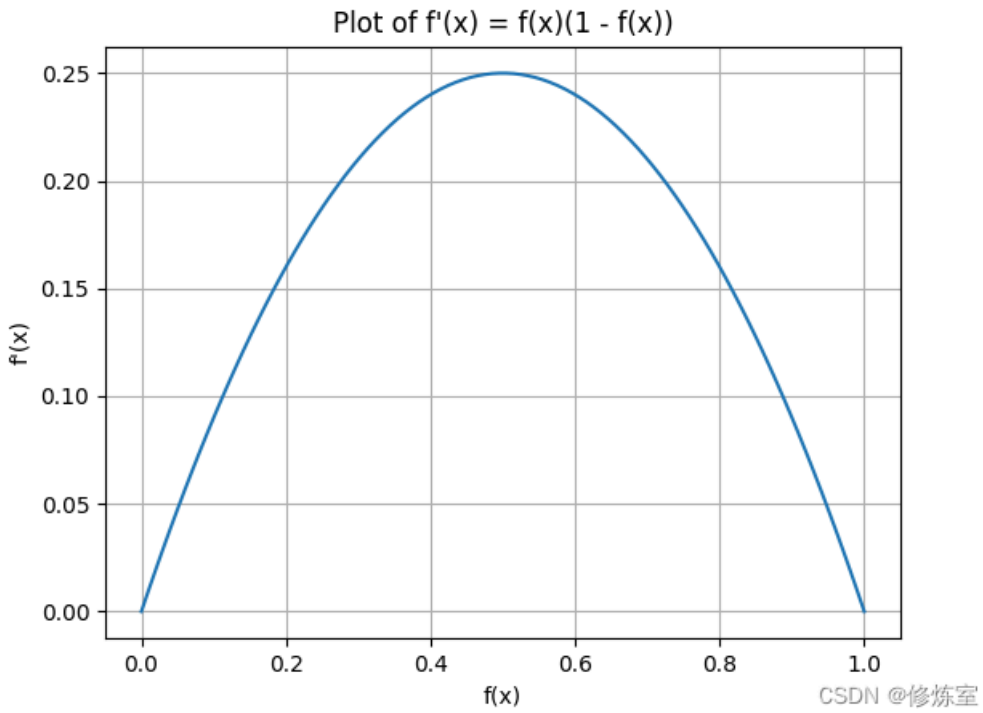
因为$h_t = f(UX_t + Wh_{t-1})$，所以我们可以得到

$$\frac{\partial h_i}{\partial h_{i-1}} = f' \times W$$

因为$f' \in [0, 0.25]$（假设为Sigmoid函数），所以说

- 如果 $W < 4$，那么连乘很多次后，导致梯度消失
- 如果 $W > 4$，那么连乘很多次后，导致梯度爆炸

> **为什么$f' \in [0, 0.25]$?**
>
> f 是Sigmoid函数，其导数 f' 的取值范围在0到0.25之间。
>
> Sigmoid函数的导数表达式为 $f'(x) = f(x)(1 - f(x))$，其中 $f(x)$ 的取值范围在0到1之间。因此，$f'(x)$ 的最大值为 0.25，在 x = 0.5 时取得。
>
> 如图所示



解决梯度消失和梯度爆炸的方法

为了缓解梯度消失和梯度爆炸问题，可以采用以下几种常见的方法：

1. **梯度裁剪（Gradient Clipping）**：
   - 将梯度的绝对值限制在某个阈值范围内，防止梯度爆炸。
   - 例如，当梯度超过某个阈值时，将其裁剪到这个阈值。
2. **正则化方法**：
   - 使用L2正则化（权重衰减）防止过度活跃的神经元。
   - 增加权重更新时的惩罚项，控制权重值不至于过大。
3. **批归一化（Batch Normalization）**：
   - 对每个时间步的隐藏状态进行归一化，稳定训练过程。
   - 通过归一化，控制每个时间步的输出范围，防止梯度过大或过小。
4. **调整激活函数**：
   - 选择适当的激活函数（如ReLU、Leaky ReLU等），防止梯度消失和爆炸。
   - 例如，Leaky ReLU 在负区间也有非零导数，避免了完全的梯度消失问题。

为什么很小的梯度无法更新权重并导致无法捕捉长期依赖关系？

当梯度非常小时，反向传播的权重更新公式：

$$\Delta W = -\eta \cdot \frac{\partial L}{\partial W}$$

梯度项 $\frac{\partial \mathcal{L}}{\partial W}$ 会非常小。这里，η 是学习率。当梯度接近零时，权重更新 ΔW 也会接近零。这意味着神经网络 的权重几乎不会发生变化，导致模型无法从训练数据中学习到有用的信息，从而无法有效捕捉长期依赖

关系。